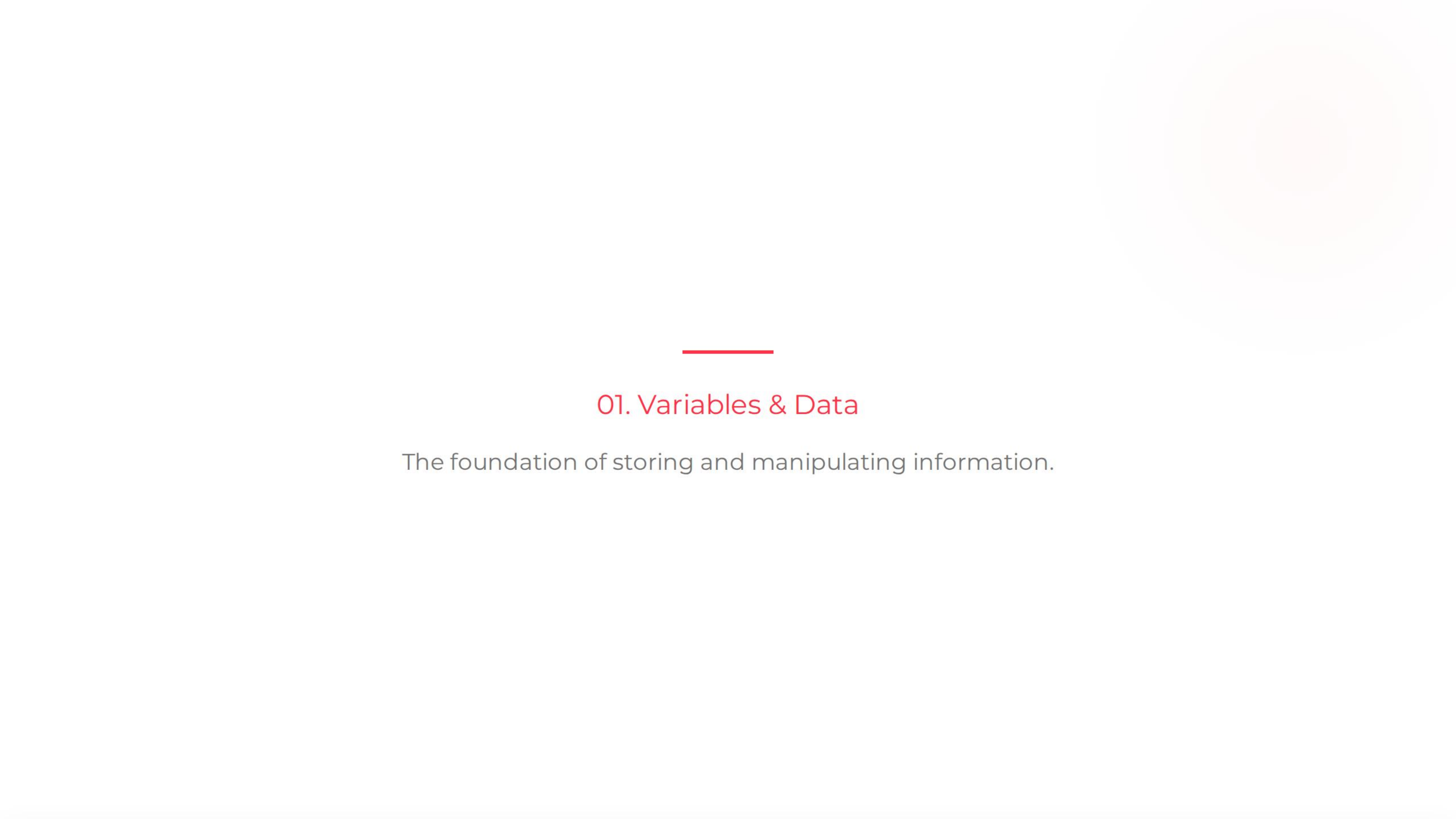


JavaScript Fundamentals

Mastering the core building blocks of modern programming.



01. Variables & Data

The foundation of storing and manipulating information.

| Variable Declaration



const

Used for values that do not change. It prevents accidental reassignments and is the default choice for modern development.



let

Allows block-scoped reassignment. Essential for counters, loops, and values that evolve over the course of execution.



var

The legacy declaration keyword. Function-scoped and subject to hoisting. Generally avoided in modern JavaScript practices.

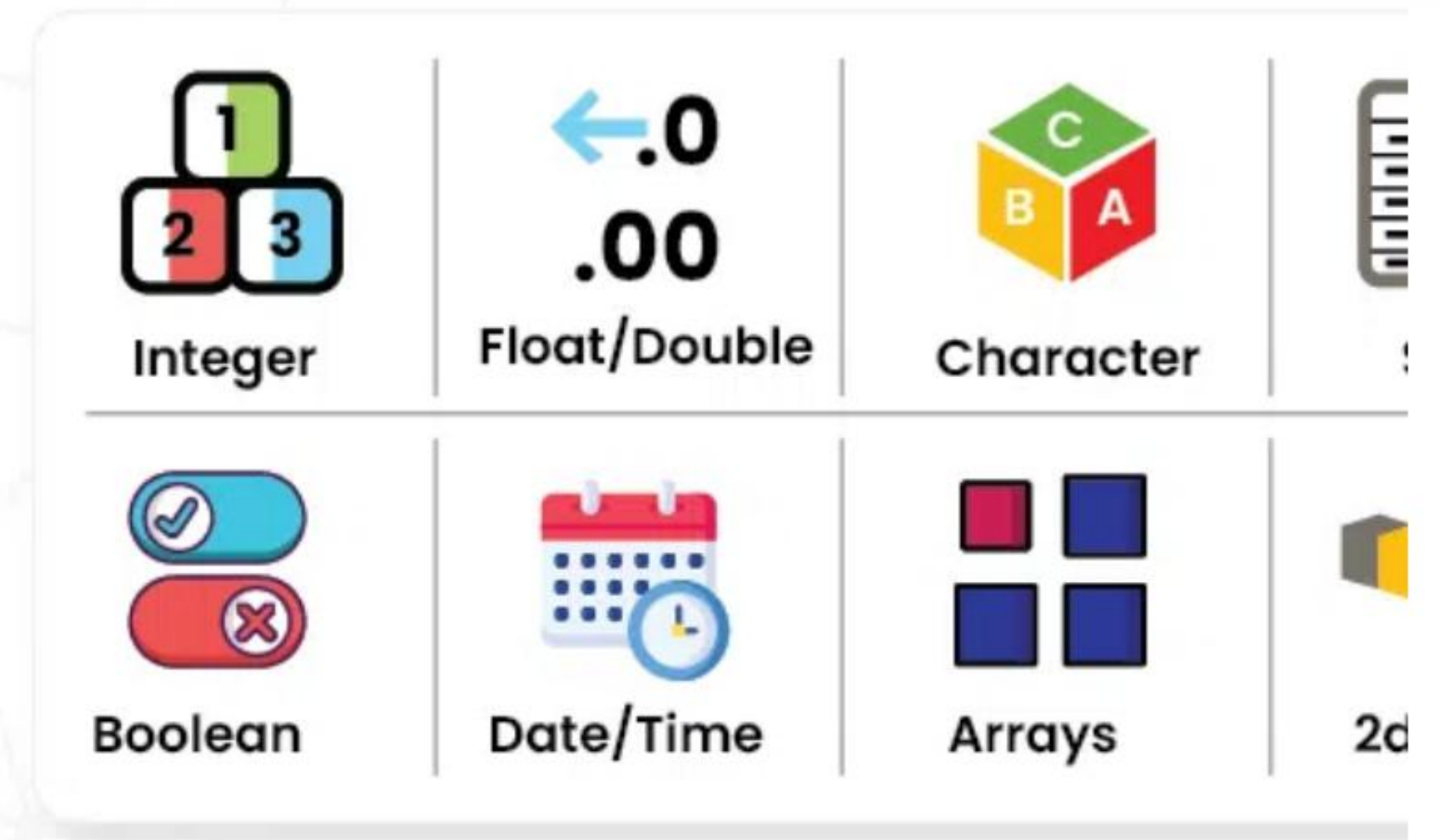
| Storing Information

Memory Management

Variables act as containers for data. When we declare a variable, JavaScript allocates space in memory to store the assigned value.

Modern engines optimize this process, ensuring that data is accessed and updated efficiently during runtime.

Types
programming

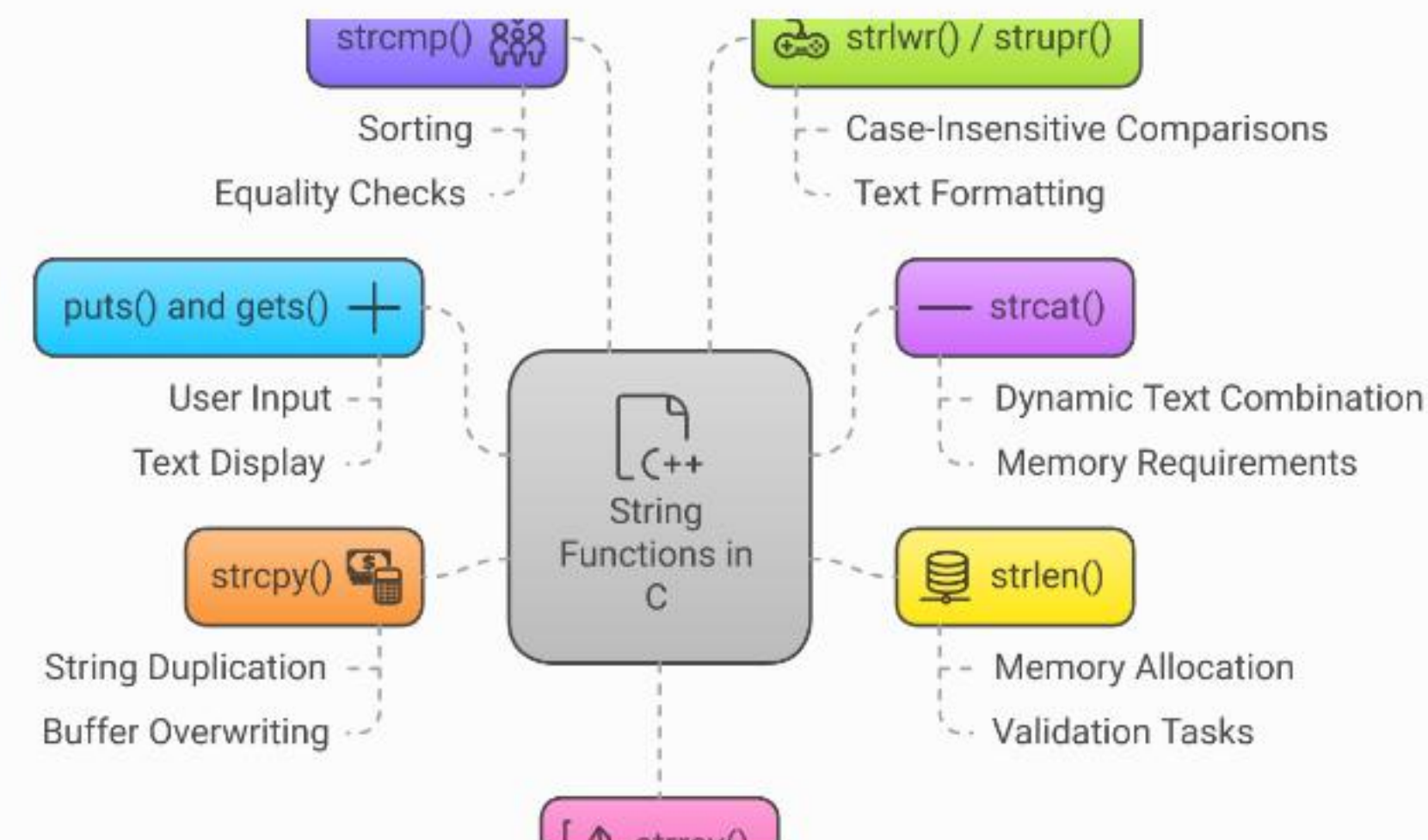


Primitive Data Types



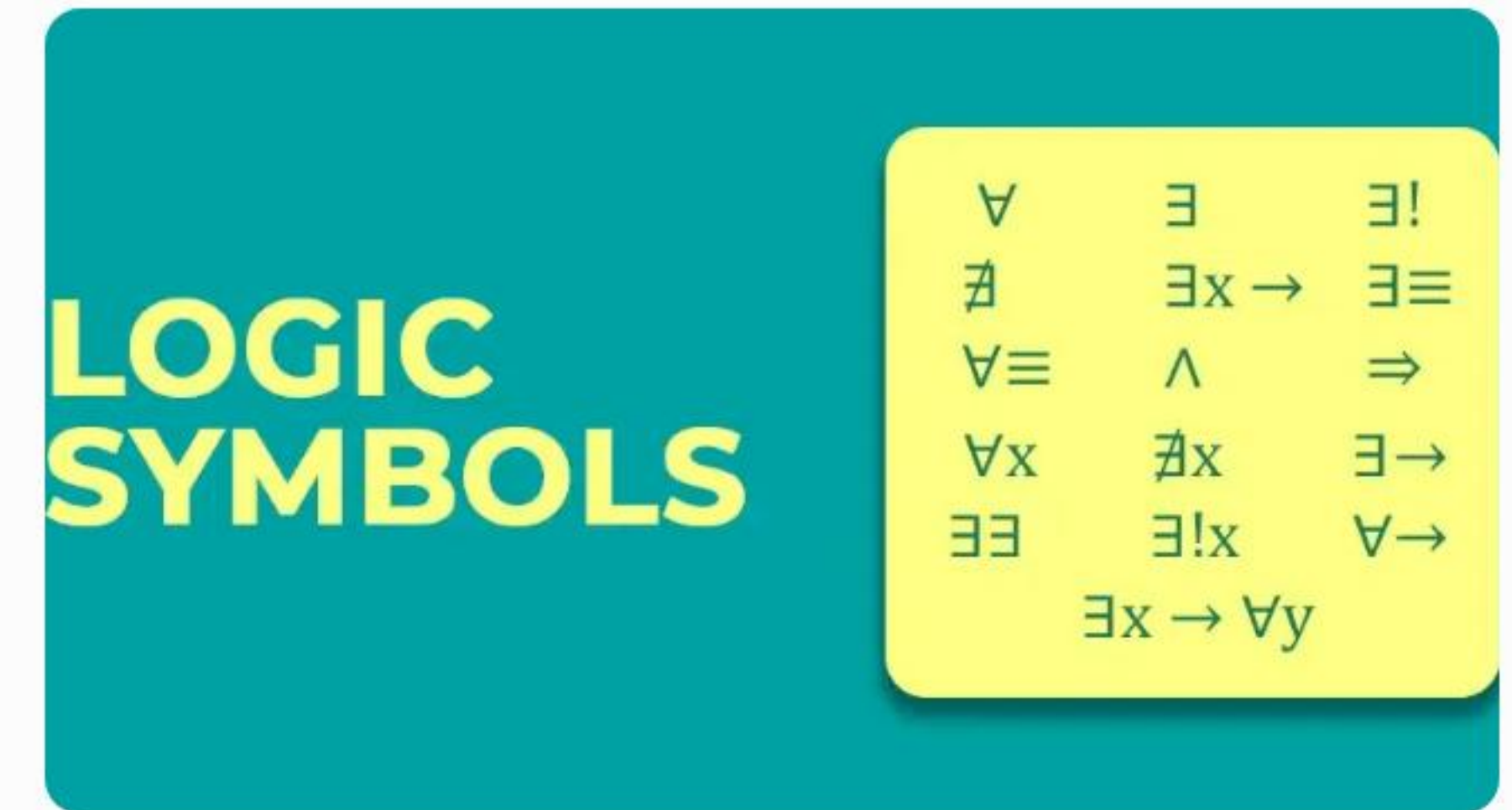
Number & String

Numbers handle all mathematical values, while Strings represent textual data wrapped in quotes.



Boolean

Represent logical entities: **true** or **false**. Essential for conditional logic and flow control.



Null & Undefined

null represents intentional absence, while **undefined** signifies a variable that has been declared but not assigned.

| String Literals

Standard Literals

Defined using single (") or double (") quotes. They are the traditional way to define strings but require escape characters for newlines.

Template Literals

Defined with backticks (` `). They allow for multi-line strings and easy expression interpolation using the `${}` syntax.

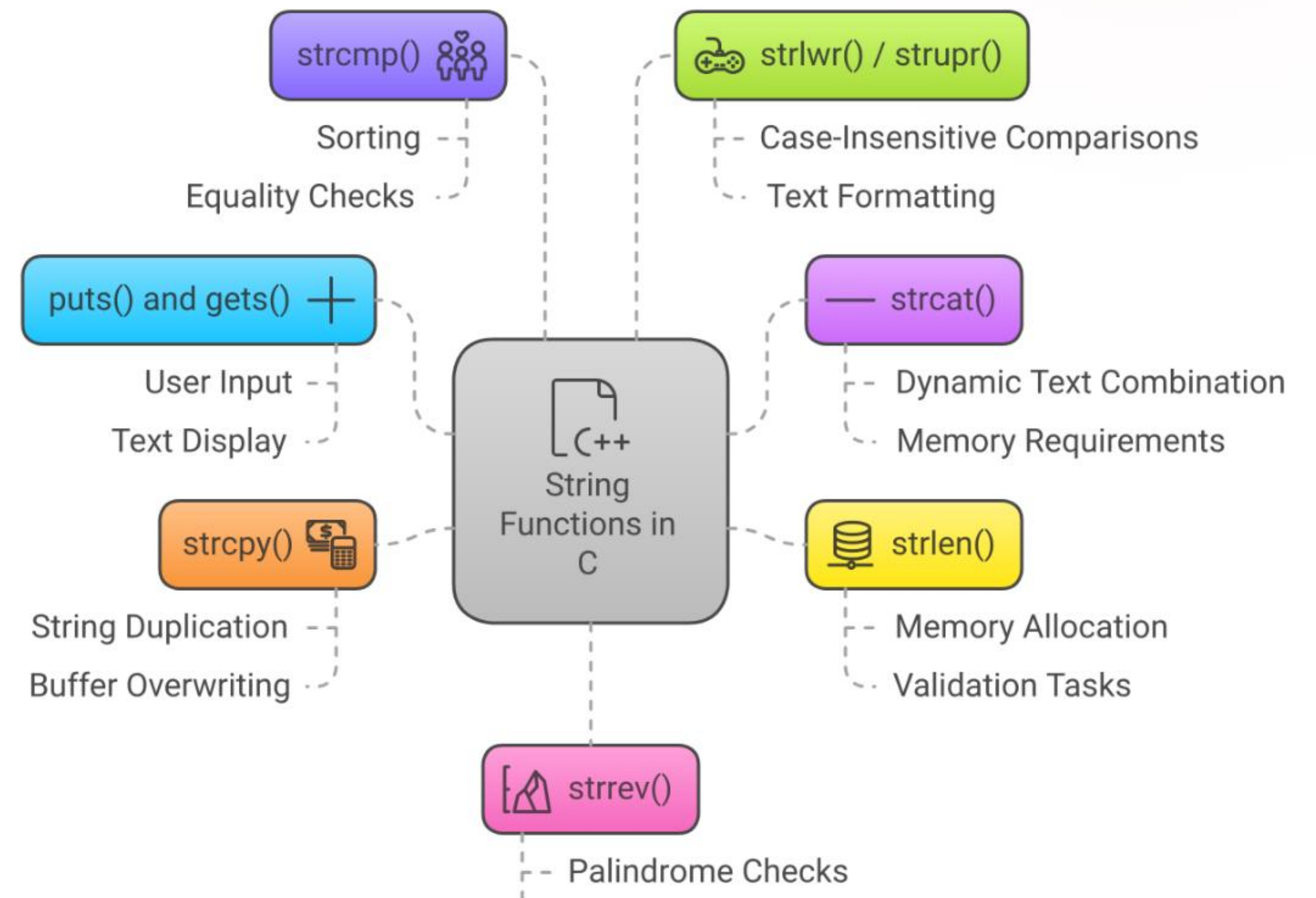
| String Concatenation

Joining Text

Concatenation combines multiple strings into one. Historically, this was done using the `+` operator.

Modern JavaScript favors template literals for concatenation as they are cleaner and less prone to spacing errors.

Example: ``Hello ${name}`` vs `"Hello " + name`.



02. Operators & Logic

Transforming data and controlling application flow.

| Types of Operators



Arithmetic

Basic math operations:
Addition (+), Subtraction (-),
Multiplication (*), and
Division (/).



Assignment

Storing values in variables
using `=`, or combining
operations like `+=` and `-=`.



Comparison

Evaluating relationships
between values: Greater
than (>), Less than (<), and
Equality (==).

| Logical Operations

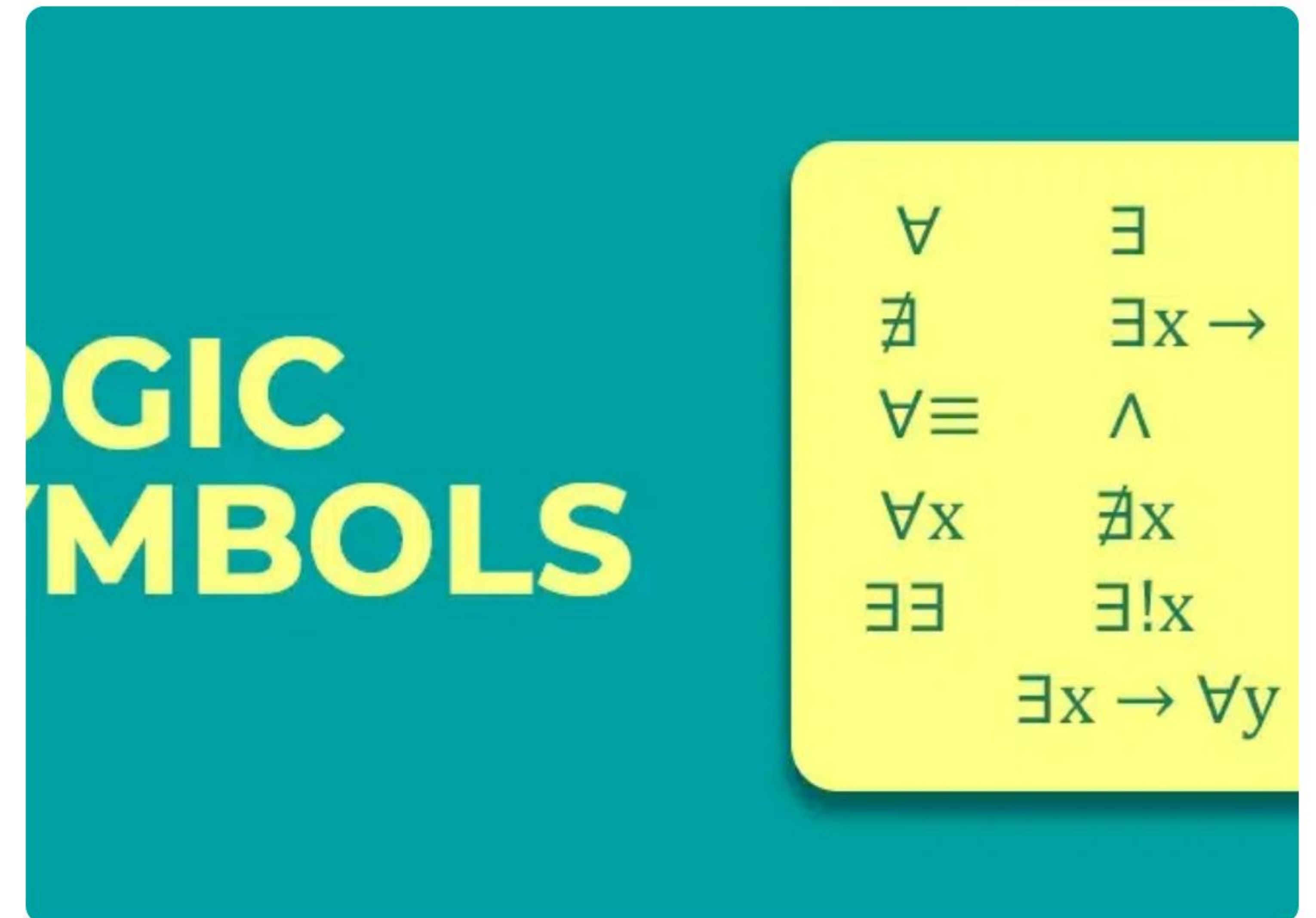
Complex Logic

Logical operators combine multiple boolean expressions to determine a final result.

AND (&&): True only if both sides are true.

OR (||): True if at least one side is true.

NOT (!): Reverses the boolean state of a value.



| Type Conversion



Type	Method	Result Example
Implicit (Coercion)	'5' + 2	'52' (String)
Explicit (String)	String(100)	'100'
Explicit (Number)	Number('42')	42
Explicit (Boolean)	Boolean(1)	true

Questions?

Thank you for exploring the basics of JavaScript.

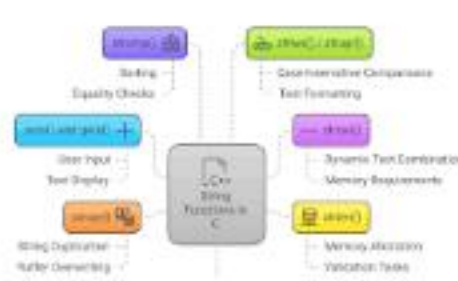
support@jsfundamentals.com | [@JSbasics](https://twitter.com/JSbasics)

| Image Sources



<https://media.geeksforgeeks.org/wp-content/uploads/20240123141843/data-types-in-programming.webp>

Source: www.geeksforgeeks.org



https://ik.imagekit.io/upgrad1/abroad-images/imageCompo/images/Top_9_Popular_String_Functions_in_C_with_Examples_Every_Programmer_Should_Know_in_2025_visual_selection8NFQXM.png?pr=true

Source: www.upgrad.com



<https://media.geeksforgeeks.org/wp-content/uploads/20240305124641/Logic-symbols-copy.webp>

Source: www.geeksforgeeks.org